

UNIVERSIDADE FEDERAL DE RORAIMA | DCC

Quick Sort

Da partição à engenharia de ordenação

Estrutura de Dados | Prof. Filipe Dwan Pereira

Nome: _____ Data: _____

Uma leitura algorítmica, histórica e experimental.

Roteiro

1. Ideia e contexto histórico
2. Partição, recursão e complexidade
3. Exemplo com seis valores
4. Variantes e comparação
5. Evidências da `lesson11`

Ideia em uma frase

Escolher um pivô, particionar a sequência em torno dele e ordenar recursivamente os lados.

Objetivo: entender o método e separar teoria de medição.

O problema de ordenação

Definição

Dada uma sequência de chaves $A = (a_0, \ldots, a_{n-1})$, produzir uma permutação A' tal que

$$a'_0 \leq a'_1 \leq \cdots \leq a'_{n-1}.$$

O conteúdo deve ser preservado; muda-se a ordem, não o multiconjunto de valores.

Propriedade de ordenação
Cada elemento aparece na posição compatível com a relação de ordem.

Propriedade de permutação
Nenhuma chave é perdida, criada ou duplicada durante o processo.

[2, 3]

História

1959: uma necessidade concreta em Moscou

A origem do Quick Sort está ligada à tradução automática e ao custo de acessar dados em fita magnética.

1959

Moscou

Universidade Estatal de Moscou

C. A. R. Hoare

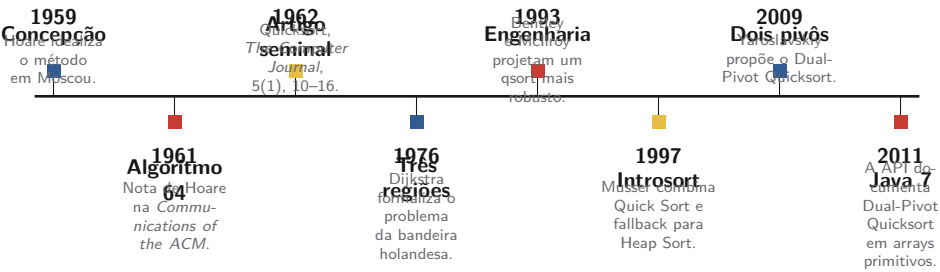
[1]

- Hoare trabalhava em um problema de tradução automática do russo para o inglês.
- As palavras de cada frase precisavam ser ordenadas para consultar um dicionário em fita magnética.
- Ordenar antes da consulta reduzia movimentos e retrocessos da fita.

A ideia surgiu em 1959; a descrição científica completa foi publicada em 1962.

De uma ideia a uma família de algoritmos

A evolução do Quick Sort ocorreu por meio de novas partições, políticas de pivô e mecanismos de proteção.



2009 é o marco da proposta de Yaroslavskiy; o Java 7 documenta a adoção da variante de dois pivôs.

Fontes: Hoare (1961, 1962); Dijkstra (1976); Bentley e McIlroy (1993); Musser (1997); Yaroslavskiy (2009); Oracle Java SE 7 API.

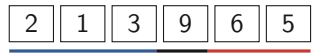
O que significa particionar?

Para um subvetor e pivô p, uma partição correta estabelece:

$$\underbrace{A[lo..q-1]}_{\leq p} \quad \underbrace{A[q]}_{p} \quad \underbrace{A[q+1..hi-1]}_{\geq p}$$

- O pivô termina em uma posição definitiva q.
- Os dois lados ainda podem estar desordenados.
- Ordenam-se os lados; não é necessário juntar (merge) depois.

[2, 4]



As relações exatas ($<$, $\leq$, etc.) dependem do esquema de partição e do tratamento de chaves iguais.

Dividir e conquistar: de onde vem o custo?

Exemplo primeiro. Para $[4, 2, 7, 1, 3, 6, 8, 5]$, usando 5 como pivô:

$$[\ \underbrace{4, 2, 1, 3}_{k=4 \text{ menores}}\ |\ \underbrace{5}_{\text{pivô}}\ |\ \underbrace{7, 6, 8}_{n-k-1=3 \text{ maiores}}\ ].$$



$$T(n) = T(k) + T(n-k-1) + \Theta(n) \qquad T(8) = T(4) + T(3) + \Theta(8)$$

Divisão equilibrada
$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$
Dois subproblemas de tamanho parecido.

Divisão extrema
$T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$
Um lado fica vazio em cada chamada.

Memória: a partição usa $O(1)$ espaço; a pilha usa $O(\log n)$ no caso equilibrado e $O(n)$ no pior.

[2, 4]

Hoare e Lomuto: duas partições

	Esquema de Hoare	Esquema Lomuto-style
Movimento	Dois índices, da esquerda e da direita.	Varredura e fronteira dos menores.
Pivô	A fronteira retornada não precisa ser a posição do pivô.	O pivô termina na fronteira.
Recursão	Usa a fronteira retornada.	Exclui a posição final do pivô.
Perfil	Geralmente faz menos trocas.	Simples; sensível a chaves iguais.

Na `lesson11`: o pivô é `values[right - 1]`; `store` avança para valores estritamente menores e, ao fim, recebe o pivô. É uma partição do tipo Lomuto.

[2, 4]

Exemplo 1: Hoare, posição e estado inicial

$$A = [1, 4, 2, 8, 5, 7], \qquad A[0] = 1,\ A[1] = 4, \ldots, A[5] = 7$$

$pivo = A[2] = 2$

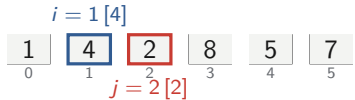
1	4	2	8	5	7
0	1	2	3	4	5

$i \leftarrow -1$, $j \leftarrow 6$. O índice i avança pela esquerda; j recua pela direita.

Regra: i para no primeiro valor $\geq pivo$; j para no primeiro valor $\leq pivo$. [1, 2]

Hoare: rodada 1, procurar e trocar

Busca de i: $A[0] = 1 < 2$, continua; $A[1] = 4 \geq 2$, para em $i = 1$.
Busca de j: $A[5] = 7 > 2$, $A[4] = 5 > 2$, $A[3] = 8 > 2$; $A[2] = 2 \leq 2$, para em $j = 2$.



$i \leq j \implies$ trocar $A[1]$ e $A[2]$

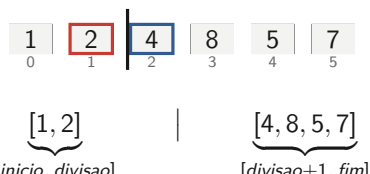


Resultado da troca: $A = [1, 2, 4, 8, 5, 7]$. O 4 estava à esquerda; o 2 estava à direita.

Hoare: rodada 2, cruzamento e recursão

Após a troca: $A = [1, 2, 4, 8, 5, 7]$, $i = 1$, $j = 2$.

- i avança: $A[2] = 4 \not< 2$, então $i = 2$.
- j recua: $A[1] = 2 \not> 2$, então $j = 1$.
- Como $i > j$, não há nova troca. A partição devolve $divisao = j = 1$.



Hoare não fixa o pivô. A função recursiva é chamada nos dois intervalos.

Hoare: chamada esquerda

Intervalo atual: $A[0..1] = [1, 2]$. Pivô:

$$pivo = A[0 + (1 - 0)/2] = A[0] = 1.$$

- $i = 0$: $A[0] = 1 \not< 1$.
- $j = 1$: $A[1] = 2 > 1$, então $j = 0$.
- $i = 0$ e $j = 0$: troca consigo mesmo; depois $i = 1$, $j = -1$.

$$[1, 2] \quad \longrightarrow \quad divisao = 0, \quad [1] \mid [2].$$

Os dois intervalos têm tamanho 1; ambas as chamadas retornam.

Hoare: chamada direita, rodada 1

Intervalo atual: $A[2..5] = [4, 8, 5, 7]$. Pivô:

$$pivo = A[2 + (5 - 2)/2] = A[3] = 8.$$

- i avança por 4 e para em $A[3] = 8$.
- j para em $A[5] = 7$, pois $7 \not> 8$.
- $i = 3 \leq j = 5$: trocar $A[3] = 8$ e $A[5] = 7$.

$$\underbrace{[1, 2, 4, 8, 5, 7]}_{\text{antes}} \xrightarrow[i=3,\ j=5]{\text{troca } A[3] \leftrightarrow A[5]} \underbrace{[1, 2, 4, 7, 5, 8]}_{\text{depois}}$$

após a troca: $\quad i \leftarrow i + 1 = 4, \qquad j \leftarrow j - 1 = 4.$

Regra: se $i < j$, troca posições diferentes; se $i = j$, a troca é consigo mesmo e o vetor não muda; se $i > j$, os índices cruzaram e a partição termina.

Os índices deixam as posições já comparadas e avançam para a próxima rodada.

Hoare: chamada direita, rodada 2

Estado: $A = [1, 2, 4, 7, 5, 8]$, intervalo $A[2..5]$.
O pivô continua sendo o elemento central do intervalo:

$$pivo = A\left[2 + \frac{5-2}{2}\right] = A[3] = 8.$$

- Após a troca anterior, $i = 4$ e $j = 4$. Então i avança: $A[4] = 5 < 8$, parando em $A[5] = 8$.
- j recua até $A[4] = 5$, pois $5 \not> 8$.
- $i = 5 > j = 4$: não há troca.

$$[4, 7, 5] \quad | \quad [8] \qquad \underbrace{divisao = j = 4}_{\text{último índice do lado esquerdo}}$$

Como $j = 4$, a divisão separa $A[2..4] = [4, 7, 5]$ de $A[5] = [8]$. O bloco [8] já é caso-base; somente $[4, 7, 5]$ continua.

Hoare: última chamada não trivial

Intervalo: $A[2..4] = [4, 7, 5]$. Pivô:

$$pivo = A[2 + (4 - 2)/2] = A[3] = 7.$$

- i passa por $A[2] = 4 < 7$ e para em $i = 3$, $A[3] = 7$.
- j para em $j = 4$, pois $A[4] = 5 \not> 7$.
- $i = 3 \leq j = 4$: trocar $A[3] = 7$ e $A[4] = 5$.

$$[4, 7, 5] \longrightarrow [4, 5, 7], \qquad i = 4,\ j = 3.$$

Agora $i = 4 > j = 3$. A divisão devolve $j = 3$: $[4, 5]$ e $[7]$. Como $[7]$ tem um único elemento, somente $[4, 5]$ ainda precisa ser analisado.

Hoare: chamada final

Intervalo atual: $A[2..3] = [4, 5]$. Pivô:

$$pivo = A[2 + (3 - 2)/2] = A[2] = 4.$$

- $i = 2$: $A[2] = 4 \not< 4$.
- $j = 3$: $A[3] = 5 > 4$, então $j = 2$.
- Troca consigo mesmo; depois $i = 3 > j = 1$.

$$[4, 5] \longrightarrow [4, 5], \qquad divisao = j = 1.$$

Os casos-base são [4] e [5].

Hoare: final da recursão

Todos os intervalos chegaram ao caso-base:

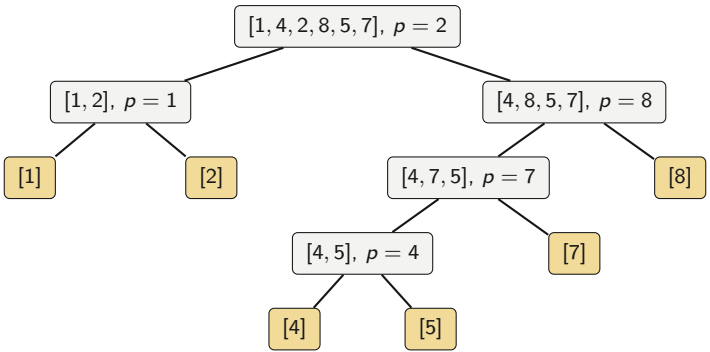
[1] | [2] | [4] | [5] | [7] | [8].



$$\boxed{A = [1, 2, 4, 5, 7, 8]}$$

O vetor está ordenado; a ordenação terminou.

Hoare: árvore de chamadas



Cada nó representa uma chamada recursiva. As folhas têm zero ou um elemento; por isso, retornam imediatamente.

Hoare divide o intervalo pelo ponto de cruzamento; o pivô não precisa aparecer como uma folha isolada.

Hoare: pseudocódigo em português

Particionar $[inicio, fim]$

1. $pivo \leftarrow A[inicio + (fim - inicio)/2]$
2. Avançar i até $A[i] \geq pivo$
   - Recuar j até $A[j] \leq pivo$
3. Se $i \geq j$, devolver j
4. Trocar $A[i]$ e $A[j]$; repetir

Ordenar

1. Se $inicio \geq fim$, retornar
2. $divisao \leftarrow$ Particionar
3. Ordenar $[inicio, divisao]$
4. Ordenar $[divisao + 1, fim]$

Ideia

Hoare devolve o ponto de cruzamento; o pivô não precisa estar pronto.

[2]

Hoare em C: partição e recursão

```c
static int particionar_hoare(int *vetor, int inicio, int fim)
{
    int pivo = vetor[inicio + (fim - inicio) / 2];
    int i = inicio - 1;
    int j = fim + 1;

    for (;;) {
        do { ++i; } while (vetor[i] < pivo);
        do { --j; } while (vetor[j] > pivo);
        if (i >= j) return j;

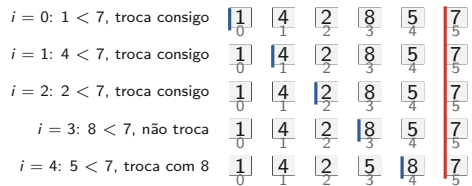
        int temporario = vetor[i];
        vetor[i] = vetor[j];
        vetor[j] = temporario;
    }
}
static void ordenar_hoare(int *vetor, int inicio, int fim)
{
    if (inicio >= fim) return;
    int divisao = particionar_hoare(vetor, inicio, fim);
    ordenar_hoare(vetor, inicio, divisao);
    ordenar_hoare(vetor, divisao + 1, fim);
}
```

`for (;;) {` repete indefinidamente; `return j` encerra o laço quando $i \geq j$. Portanto, ele não fica sem saída.

Chamada inicial: `ordenar_hoare(vetor, 0, n - 1)`.

Exemplo 2: Lomuto, cada leitura

$A = [1, 4, 2, 8, 5, 7]$, índices $0, 1, \ldots, 5$.
Pivô: $pivo = A[5] = 7$. Início: $fronteira = 0$.

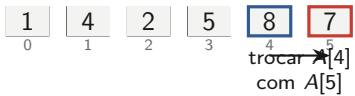


Azul: posição da fronteira antes da leitura. Vermelho: pivô. Troca consigo não altera o vetor, mas é executada pela versão literal do código. [2, 4]

Lomuto: fixar o pivô e continuar

Depois de $i = 4$:

$$A = [1, 4, 2, 5, 8, 7], \qquad \textit{fronteira} = 4.$$



$$[1, 4, 2, 5, 7, 8]$$

- O laço já processou $A[0]$ até $A[4]$. Os valores $1, 4, 2, 5$ são menores que 7; por isso, *fronteira* $= 4$. O valor $A[4] = 8$ não é menor que o pivô.
- A fronteira indica o lugar reservado para o pivô: trocar $A[4] = 8$ com $A[5] = 7$.
- Depois da troca, 7 fica na posição definitiva 4: $[1, 4, 2, 5] \mid [7] \mid [8]$.
- Próximos intervalos: $[0, 4)$ e $[5, 6)$; a recursão continua apenas em $[0, 4)$, pois $[5, 6)$ tem um elemento.

Lomuto: segunda chamada

Intervalo esquerdo: $A[0..3] = [1, 4, 2, 5]$.
Pivô: $pivo = A[3] = 5$, $fronteira = 0$.

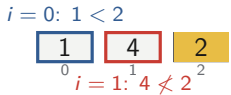
- $i = 0$: $1 < 5$; troca consigo; $fronteira = 1$.
- $i = 1$: $4 < 5$; troca consigo; $fronteira = 2$.
- $i = 2$: $2 < 5$; troca consigo; $fronteira = 3$.
- Troca final $A[3]$ com $A[3]$; o vetor não muda.

$$[1, 4, 2, 5] \quad \longrightarrow \quad [1, 4, 2] \mid [5]$$

O pivô 5 já está na posição definitiva 3; resta ordenar $[1, 4, 2]$.

Lomuto: terceira chamada

Intervalo atual: $A[0..2] = [1, 4, 2]$.
Pivô: $pivo = A[2] = 2$, $fronteira = 0$.



- $i = 0$: troca consigo; $fronteira = 1$.
- $i = 1$: não troca.
- Troca final $A[1] = 4$ com $A[2] = 2$.

$$[1, 2, 4, 5, 7, 8] \qquad \text{posição do pivô} = 1$$

Lomuto: final da recursão

Depois da terceira chamada:

$$[1] \mid [2] \mid [4] \mid [5] \mid [7] \mid [8].$$

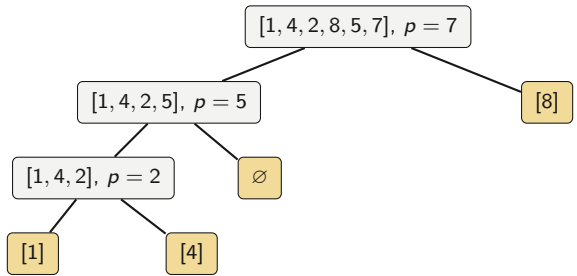
Cada intervalo restante tem tamanho 0 ou 1, portanto a função retorna.



$$\boxed{A = [1, 2, 4, 5, 7, 8]}$$

Lomuto fixa um pivô por partição; a recursão termina quando não há mais de um elemento.

Lomuto: árvore de chamadas



Cada pivô fica em sua posição definitiva antes das chamadas filhas. Intervalos vazios também são casos-base.

A árvore mostra a ordem estrutural: particionar, ordenar à esquerda e depois à direita.

Lomuto: pseudocódigo em português

Particionar $[inicio, fim)$

1. $pivo \leftarrow A[fim - 1]$; $fronteira \leftarrow inicio$
2. Para i de $inicio$ a $fim - 2$:
   - se $A[i] < pivo$, trocar com $A[fronteira]$
   - avançar `fronteira`
3. Trocar $A[fronteira]$ e o pivô
4. Devolver $fronteira$

Ordenar

1. Se $fim - inicio < 2$, retornar
2. $posicao \leftarrow$ Particionar
3. Ordenar o intervalo à esquerda
4. Ordenar o intervalo à direita

Lomuto devolve a posição definitiva do pivô.

[2]

Lomuto em C: partição e recursão

```c
static size_t particionar_lomuto(int *vetor, size_t inicio, size_t fim)
{
    int pivo = vetor[fim - 1];
    size_t fronteira = inicio;

    for (size_t indice = inicio; indice + 1 < fim; ++indice) {
        if (vetor[indice] < pivo) {
            int temporario = vetor[indice];
            vetor[indice] = vetor[fronteira];
            vetor[fronteira] = temporario;
            ++fronteira;
        }
    }
    int temporario = vetor[fronteira];
    vetor[fronteira] = vetor[fim - 1];
    vetor[fim - 1] = temporario;
    return fronteira;
}

static void ordenar_lomuto(int *vetor, size_t inicio, size_t fim)
{
    if (fim - inicio < 2) return;
    size_t posicao = particionar_lomuto(vetor, inicio, fim);
    ordenar_lomuto(vetor, inicio, posicao);
    ordenar_lomuto(vetor, posicao + 1, fim);
}
```

Chamada inicial: `ordenar_lomuto(vetor, 0, n)`.

Contagem do exemplo: operações e tempo

Convenção: $A[0]$ é o primeiro elemento; $A[5]$ é o sexto. Nesta execução, contamos somente a primeira partição.

	Hoare	Lomuto	O que foi contado
Comparações com o pivô	8	5	Testes como $A[i] < pivo$ e $A[j] > pivo$.
Verificações $i \geq j$	2	–	Exclusivas do esquema de Hoare.
Trocas executadas	1	5	Lomuto faz 3 trocas consigo mesmo.
Trocas que mudam o vetor	1	2	Hoare: $4 \leftrightarrow 2$; Lomuto: $8 \leftrightarrow 5$ e $8 \leftrightarrow 7$.

$$\text{tempo medido} = \text{instante final} - \text{instante inicial} \quad \neq \quad \text{número de operações}$$

A contagem explica o custo; o tempo em milissegundos depende do relógio, compilador, máquina e execução.

Propriedades: o que o Quick Sort oferece (e não oferece)

Pontos fortes

- Médio: $\Theta(n \log n)$.
- In-place e acesso sequencial ao vetor.
- Código compacto e adaptável.

Limitações

- Pior caso: $\Theta(n^2)$.
- Pilha pode chegar a $O(n)$.
- Não é estável na forma usual.
- Pivô e chaves iguais importam.

In-place não significa memória total $O(1)$: a pilha também ocupa espaço.

[2, 3, 4]

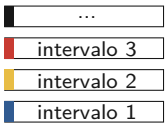
Memória: o que significa in-place?

Vetor original

$$\underbrace{\boxed{\text{dados}}}_{O(n)} + \underbrace{\boxed{\text{trocas}}}_{O(1)}$$

- A partição rearranja o próprio vetor.
- Não há vetor auxiliar de tamanho n.

Pilha de chamadas



equilibrada: $O(\log n)$ ou extrema: $O(n)$

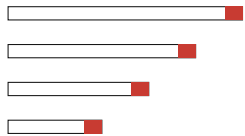
[2, 4]

Quando o pivô dá errado?

Entrada já ordenada + maior pivô

$$(n-1)+(n-2)+\cdots+1 = \frac{n(n-1)}{2} = \Theta(n^2)$$

Um lado fica vazio em cada chamada.

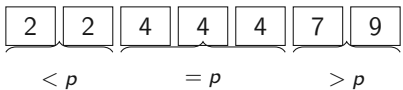


um lado quase vazio
a cada partição

Soluções: pivô aleatório, mediana, introsort.

Chaves repetidas: três vias

Separar em três regiões: $< p$, $= p$ e $> p$. A região igual não precisa ser recursada.



- Dijkstra mantém fronteiras para as três regiões.
- A região $= p$ não é recursada.
- A `lesson11` usa Lomuto, não três vias.

[5, 6, 4]

Variantes do método

Técnica	O que busca resolver
Pivô aleatório	Diminui a chance de entrada adversa fixa.
Mediana de três	Evita pivôs extremos em padrões comuns.
Três vias	Reduz recursão quando há muitos iguais.
Inserção em partes pequenas	Reduz custo constante e chamadas curtas.
Introsort	Usa Heapsort como proteção contra n^2.
Dual-pivot	Usa duas fronteiras; depende da implementação.

Exemplo: Java documenta Dual-Pivot Quicksort para certas ordenações de arrays primitivos.

[7, 6, 10]

Comparação com outros algoritmos do trabalho

Algoritmo	Tempo típico / pior	Estável?	In-place?	Observação principal
Quick Sort	$n \log n$ / n^2	Não	Em geral	Rápido na prática; o pivô importa.
Merge Sort	$n \log n$ / $n \log n$	Sim*	Não*	Garantia de pior caso; memória auxiliar.
Heap Sort	$n \log n$ / $n \log n$	Não	Sim	Pior caso garantido; acesso menos sequencial.
Insertion Sort	n^2 / n^2	Sim	Sim	Simples; útil em entradas pequenas/quase ordenadas.
Selection Sort	n^2 / n^2	Não*	Sim	Poucas trocas; muitas comparações.
Bubble Sort	n^2 / n^2	Sim*	Sim	Didático; depende da otimização para parar cedo.
Radix Sort	$d(n+b)$ / $d(n+b)$	Depende	Não*	Não compara chaves; custo depende da representação e base.

$*$ Propriedades da versão usual/implementação estável, não de todo algoritmo que compartilha o nome. d: número de dígitos/passagens; b: base ou tamanho do alfabeto.

[2, 8, 3]

Busca: o que muda quando os dados estão ordenados?

Busca sequencial

Examina itens até encontrar a chave ou chegar ao fim.

Pior caso: $O(n)$. Não exige ordenação.

Busca binária

Compara com o elemento central e descarta metade do intervalo a cada etapa.

Pior caso: $O(\log n)$. Exige sequência ordenada e acesso indexado eficiente.

Na `lesson11`, o benchmark de buscas usa 10.000 chaves e 10.000 consultas pseudoaleatórias: sequencial no vetor original e binária em uma cópia ordenada.

[2]

Metodologia experimental da `lesson11`

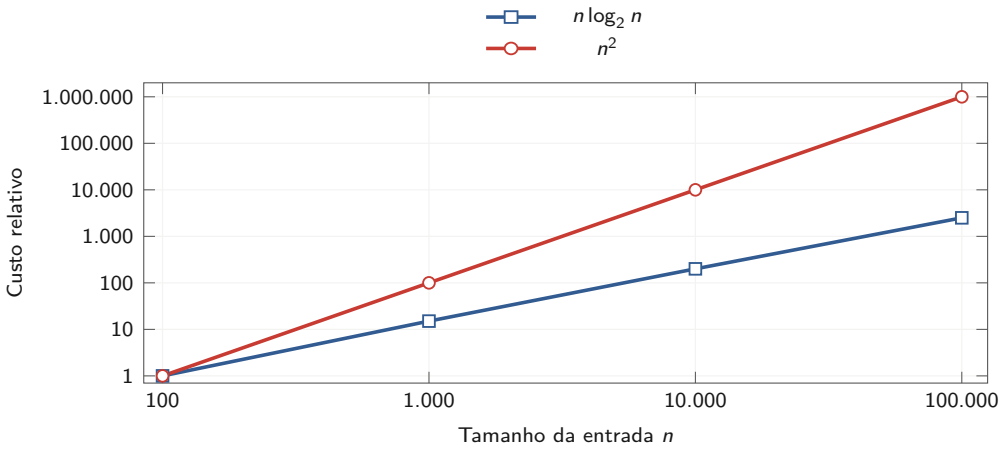
- Entradas: 100, 1.000, 10.000 e 100.000 inteiros.
- Mede apenas a chamada do algoritmo; gravar o arquivo fica fora do tempo.
- Cada saída foi conferida: ordem não decrescente e mesmos valores da entrada.
- Os nomes guardam tempos arredondados; não há repetições nem dispersão.

Escopo observado

Há artefatos completos para as sete ordenações nos quatro tamanhos: 100, 1.000, 10.000 e 100.000 elementos. Cada saída foi conferida quanto à ordenação e à preservação do multiconjunto de entrada.

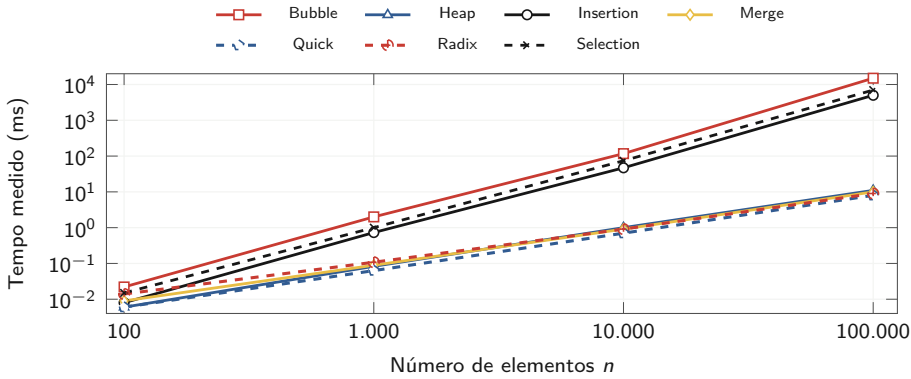
Fonte: `problems/lesson11/problem_110.c`, `search-benchmark.c` e `data/`.

Teoria: forma da curva



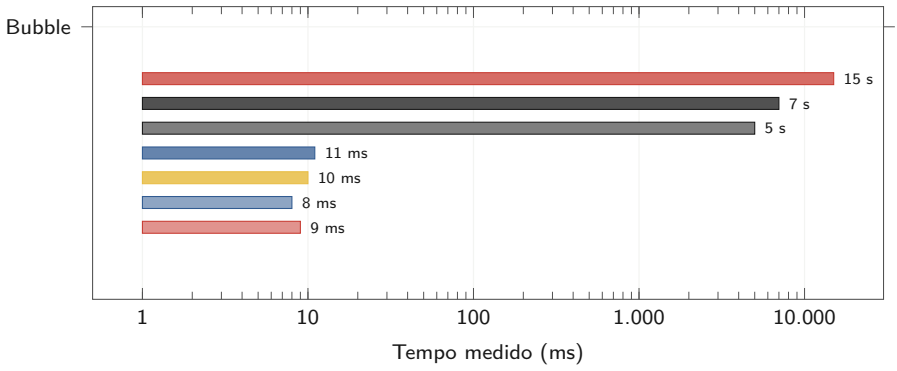
Curvas normalizadas em $n = 100$. Não são tempos medidos.

Resultados práticos registrados: crescimento por tamanho



Eixos em escala log–log: cada marca no eixo y é uma potência de dez ($10^0 = 1$ ms). Assim, 15.000 ms aparece na mesma figura que 0,006 ms.

100.000 elementos: comparação direta



Escala logarítmica: a barra mostra a ordem de grandeza, não uma distância linear.

Como o crescimento aparece nos dados

Modelos de crescimento

Modelo	100	1.000	10.000	100.000
$n \log_2 n$	1	15,1	200,1	2.501
n^2	1	100	10.000	1.000.000
n	1	10	100	1.000

Para comparar apenas a forma, todos começam em 1 quando $n = 100$.

Medições relativas

Algoritmo	100	1.000	10.000	100.000
Quick	1	10,5	116,8	1.333,3
Heap	1	13,8	166,7	1.833,3
Merge	1	9,9	101,2	1.111,1
Radix	1	7,7	65,1	642,9

Base: o menor arquivo de cada algoritmo; valores aproximados pelos nomes dos arquivos.

Como ler: a tabela da esquerda mostra três formas de crescimento; a da direita mostra o que ocorreu nesta execução. Quick, Heap e Merge ficaram próximos do padrão $n \log n$; Radix cresceu menos nesta faixa.

O que cada curva sugere

Algoritmo	Custo conhecido	Medição	Leitura desta execução
Quick Sort	Médio $n \log n$; pior n^2	$0,006 \rightarrow 8$ ms	Acompanhou $n \log n$ nesta entrada.
Heap Sort	$n \log n$	$0,006 \rightarrow 11$ ms	Crescimento esperado; pior caso garantido.
Merge Sort	$n \log n$	$0,009 \rightarrow 10$ ms	Crescimento esperado.
Radix Sort	$d(n + b)$	$0,014 \rightarrow 9$ ms	Próximo de linear nesta faixa.
Insertion Sort	n^2	$0,008 \rightarrow 5$ s	Salto compatível com custo quadrático.
Selection Sort	n^2	$0,015 \rightarrow 7$ s	Aumento acentuado até 100.000.
Bubble Sort	n^2	$0,022 \rightarrow 15$ s	Maior tempo observado.

"Compatível" significa que os pontos não contradizem a ordem de crescimento esperada; não é uma prova estatística da complexidade.

Como ler os resultados

1. **Teoria:** compara crescimento; não prevê milissegundos.
2. **Prática:** depende de hardware, código, entrada e arredondamento.
3. **Quick, Heap e Merge:** curvas próximas, como esperado para $n \log n$.
4. **Bubble, Insertion e Selection:** aumento acentuado, compatível com n^2.
5. **Radix:** depende de dígitos, base e representação da chave.

Mensagem para a banca

Os dados são coerentes com as previsões assintóticas nas entradas observadas, mas a conclusão deve ser limitada ao ambiente, às entradas e aos arquivos efetivamente medidos.

Leitura dos resultados: o que os dados sustentam

Algoritmo	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$	Leitura cuidadosa
Quick Sort	6 μs	63 μs	701 μs	8 ms	Menor tempo registrado nos quatro tamanhos, empatado com Heap em $n = 100$.
Heap Sort	6 μs	83 μs	1 ms	11 ms	Crescimento compatível com $n \log n$.
Merge Sort	9 μs	89 μs	911 μs	10 ms	Mesmo patamar assintótico de Quick e Heap.
Insertion Sort	8 μs	730 μs	47 ms	5 s	Crescimento acentuado, compatível com custo quadrático.
Selection Sort	15 μs	1 ms	74 ms	7 s	Aumento acentuado até 100.000.
Bubble Sort	22 μs	2 ms	117 ms	15 s	Maior tempo registrado entre os sete algoritmos.
Radix Sort	14 μs	108 μs	911 μs	9 ms	Melhor tempo registrado em 100.000 nesta amostra.

Nomes consultados: `100000_15s.txt`, `100000_5s.txt`, `100000_7s.txt`, `100000_11ms.txt`, `100000_10ms.txt`, `100000_8ms.txt` e `100000_9ms.txt`.

O que não se pode concluir apenas com esses tempos

- Uma execução por algoritmo/tamanho não estima variabilidade nem intervalo de confiança.
- A resolução dos nomes varia entre microssegundos, milissegundos e segundos; os valores são arredondados.
- Não foram registrados aqui CPU, sistema, compilador, flags, estado de cache ou distribuição detalhada das entradas.
- Quick Sort é o menor tempo *registrado* nesta amostra, mas isso não prova superioridade universal.

Como fortalecer o experimento

Executar várias repetições; usar as mesmas entradas e cópias independentes; relatar mediana e dispersão; registrar compilador/flags e hardware; testar entradas aleatórias, ordenadas, reversas e com muitos valores iguais.

Por que ainda escolher Quick Sort?

É uma boa escolha quando

- dados cabem em memória;
- estabilidade não é requisito;
- desempenho médio e localidade são prioritários;
- a implementação pode escolher boas políticas de pivô e fallback.

Considere alternativas quando

- pior caso garantido é obrigatório (Heap/Merge/Introsort);
- estabilidade é obrigatória (Merge);
- dados são quase ordenados e pequenos (Insertion);
- a chave tem estrutura explorável (Radix).

A escolha é uma decisão de engenharia: modelo de dados, requisitos e implementação importam tanto quanto a notação assintótica.

[3, 2, 8]

Conexão com o trabalho da disciplina

- Integra o menu de comparação da `lesson11`.
- Gera arquivos ordenados em `data/quick-sort/`.
- Há $7 \times 4 = 28$ resultados válidos.

Fontes: `problems/lesson11/quick-sort.c`, `problem_110.c` e `data/quick-sort/`.



Substituir por uma captura da tela de comparação ou do menu da `lesson11`.

Síntese

1. Particionar é dividir; depois, ordenar os dois lados.
2. Partições equilibradas dão $n \log n$; partições extremas dão n^2.
3. A `lesson11` usa pivô final e Lomuto.
4. Pivô aleatório, três vias e introsort tratam riscos diferentes.
5. Nesta amostra, Quick Sort teve o menor tempo registrado.

Obrigado(a)!
Perguntas?

Referências I


HOARE, C. A. R. Quicksort. *The Computer Journal*, v. 5, n. 1, p. 10–16, 1962. doi:10.1093/comjnl/5.1.10.


CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.


KNUTH, D. E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Addison-Wesley, 1998.


SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011. Material de apoio: `https://algs4.cs.princeton.edu/23quicksort/`.


DIJKSTRA, E. W. *A Discipline of Programming*. Prentice-Hall, 1976.


BENTLEY, J. L.; McILROY, M. D. Engineering a sort function. *Software: Practice and Experience*, v. 23, n. 11, p. 1249–1265, 1993. doi:10.1002/spe.4380231105.

Referências II


MUSSER, D. R. Introspective sorting and selection algorithms. *Software: Practice and Experience*, v. 27, n. 8, p. 983–993, 1997. doi:10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#.


ZIVIANI, N. *Projeto de Algoritmos: com implementações em Pascal e C*. 3. ed. Cengage Learning, 2011.


MARTINEZ, C. Quicksort. In: BLACK, P. E. (ed.). *Dictionary of Algorithms and Data Structures*. NIST, 2023. Disponível em: `https://xlinux.nist.gov/dads/HTML/quicksort.html`.


ORACLE. *Java Platform, Standard Edition 8 API Specification: Arrays*. Nota de implementação sobre Dual-Pivot Quicksort. `https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html`.


CARAUMA, J. A. Repositório *art-mondrian*, módulo `problems/lesson11`. Código e dados experimentais da implementação. Disponível em: `https://github.com/carauma7/art-mondrian`.